Java's Framework Collection

Bib: *Java: An Introdution to Problem Solving & Programming, Walter Savitch*

Cap. 12: Dynamic Data Structures and Generics

Tutoriais Java: https://docs.oracle.com/javase/tutorial/java/generics/index.html

Algoritmia e Estruturas de Dados (AED)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEIC, ADEETC, ISEL

ISEL

Resumo

- *Framework* de coleções Java / Estruturas de dados dinâmicas

- Complexidade de um algoritmo

Resumo

- *Framework* de coleções Java / Estruturas de dados dinâmicas

- Complexidade de um algoritmo

Estruturas de dados dinâmicas – o que é?

- Uma estrutura de dados dinâmica suporta a coleção de um número variável de elementos durante o seu tempo de vida
  - Não está sujeita a uma capacidade limite

- Um *array* é uma estrutura de dados estática
  - Durante o seu tempo de vida não é possível alterar a capacidade limite

- No JavaSE, versão 2, foi definida uma hierarquia de interfaces e classes que constituem, desde então, a *framework de coleções Java (Java's Collection Framework)*
  - Ao longo de várias versões esta *framework* tem evoluído, nomeadamente, a introdução de genéricos, na versão 5

Diagrama de classes da *Java's Collection Framework* (1 de 2)

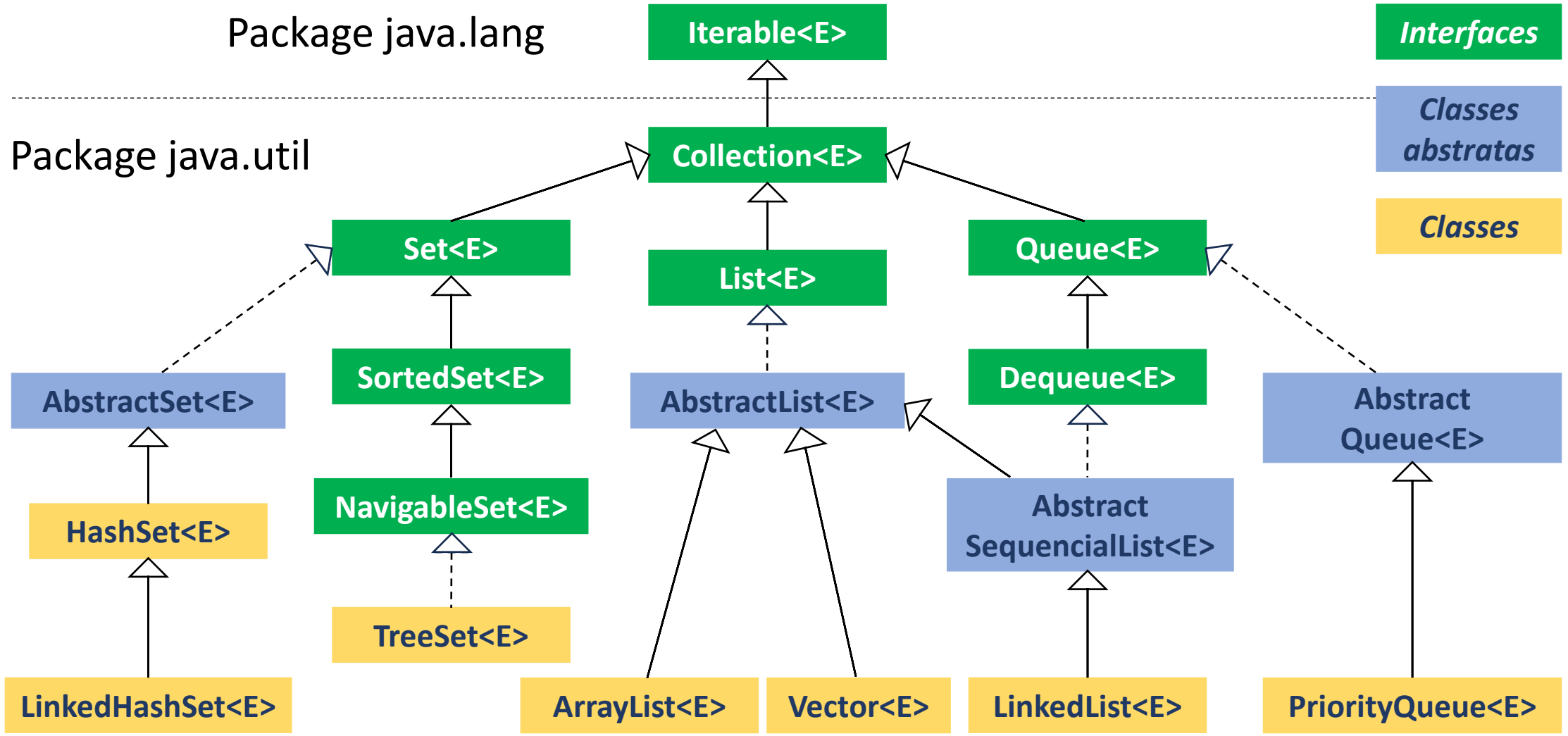
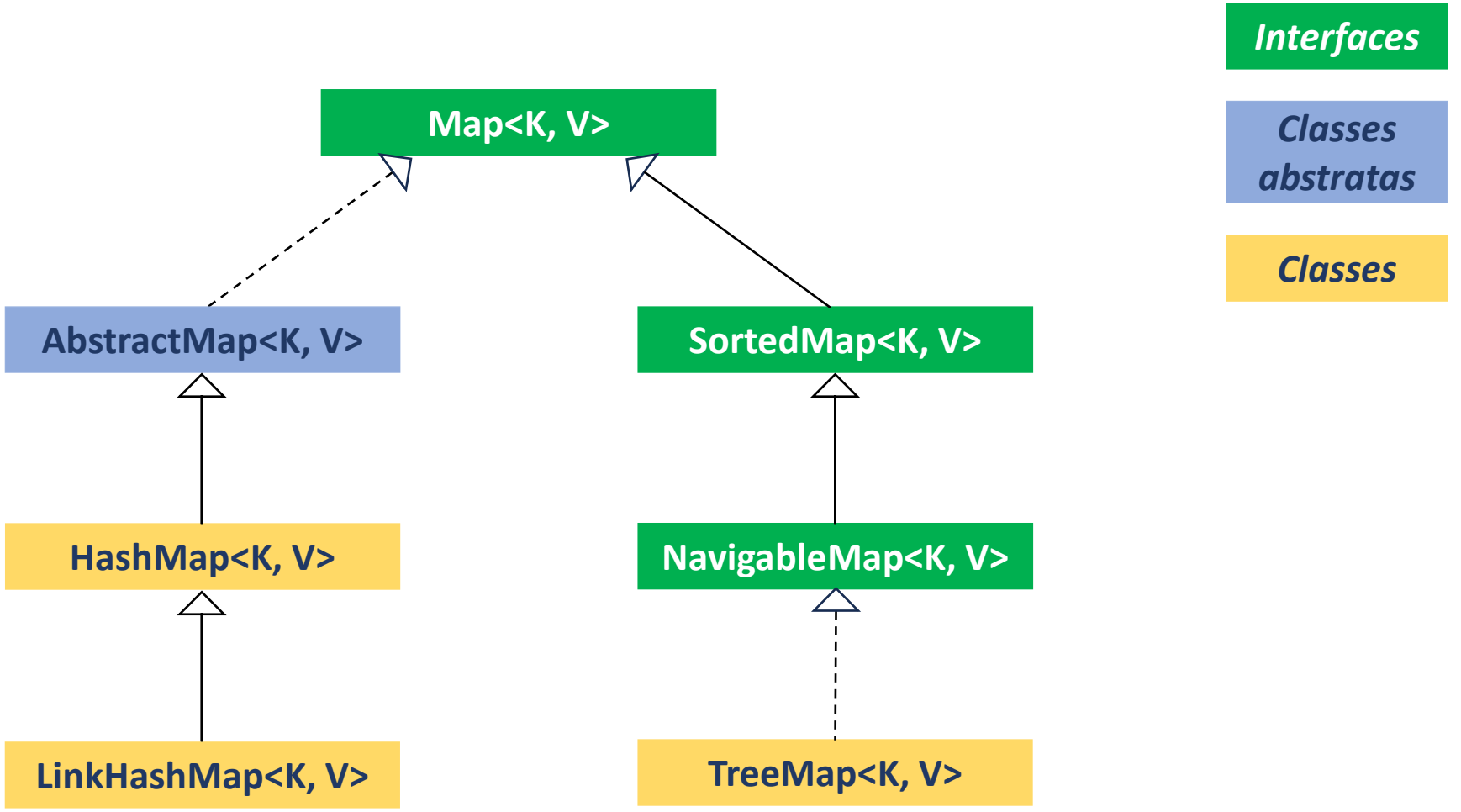


Diagrama de classes da *Java's Collection Framework* (2 de 2)

Package java.util



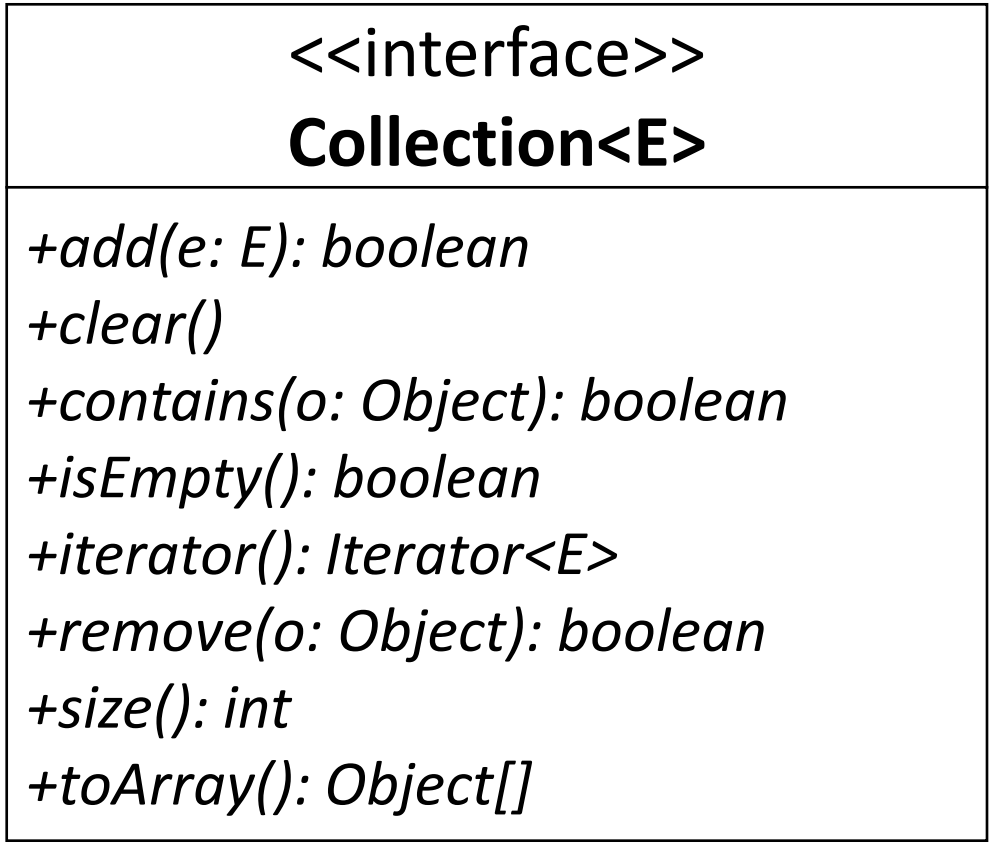
Visão geral da *framework* de coleções Java

- Infraestrutura implementada no package `java.util`
- Conjunto de interfaces e classes parametrizadas (ou genéricas) úteis para trabalhar com grupo de objetos, e providenciam um conjunto de ferramentas para manipular e organizar dados eficientemente
- As coleções caracterizam-se pela interface pública que disponibilizam e pelas estruturas de dados que usam para manter os elementos nas coleções: *arrays*, listas, árvores, tabelas de *hash*

- A interface `Collection` é a interface raiz de todas as coleções e `Collection` é iterável (estende `Iterable`)
- No caso de coleções que envolvem pares chave/valor, a interface raiz é a interface `Map`

Interface `Collection<E>`

- A interface `Collection` inclui mais métodos para além dos apresentados
- Parâmetros do tipo `Object` permitem uma utilização do método mais genérica, nomeadamente a utilização de objetos de tipos que não correspondam ao tipo parametrizado, usado na criação da coleção (ex: `contains` e `remove`)
- Interfaces especializadas de `Collection` incluem métodos adequados ao tipo de estruturas de dados que servem
- Uma coleção é iterável, ou seja, permite obter um `Iterador` para percorrer todos os elementos da coleção



Exemplo de uma coleção de objetos do tipo `Student` (1 de 2)

```
import java.util.ArrayList;
import java.util.Collection;

public class CollectionDemo {
    private Collection<Student> students = new ArrayList<>();

    public boolean addStudent(Student s) { return students.add(s); }
    public boolean removeStudent(Student s) { return students.remove(s); }
    public boolean containsStudent(Student s) { return students.contains(s); }
    public int getTotalStudents() { return students.size(); }
    public void clearAllStudents() { students.clear(); }
    public void printAllStudents() { students.forEach(s -> System.out.println("\t: " + s)); }
}
```

Expressão lambda

Lê-se: para cada elemento (`s`) da coleção `students` executa o corpo da expressão (`System.out...`)

Exemplo de uma coleção de objetos do tipo `Student` (2 de 2)

```
public final class Student {
    private String name;
    private int number;

    public Student(String name, int number) { this.name = name; this.number = number; }
    public String getName() { return name; }
    public int getNumber() { return number; }
    public boolean equals(Object o) {
        if (!(o instanceof Student student)) return false;
        return number == student.number;
    }
    public int hashCode() { return number; }
    public String toString() { return "name: \"" + name + "\", number: " + number; }
}
```

Método relevante para os métodos `remove` e `contains` definidos na interface `Collection`

Método relevante quando o objeto for inserido numa coleção com uma chave

Contrato genérico do método `hashCode`

- Por norma, quando se sobrepõe o método `equals` deve-se sobrepor o método `hashCode`, de forma a garantir que o contrato permanece válido

- Retirado de: https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html#hashCode--

"Whenever it is invoked on the same object more than once during an execution of a Java application, the hashCode method must consistently return the same integer, provided no information used in equals comparisons on the object is modified. This integer need not remain consistent from one execution of an application to another execution of the same application.

If two objects are equal according to the equals(Object) method, then calling the hashCode method on each of the two objects must produce the same integer result.

It is not required that if two objects are unequal according to the equals(java.lang.Object) method, then calling the hashCode method on each of the two objects must produce distinct integer results. However, the programmer should be aware that producing distinct integer results for unequal objects may improve the performance of hash tables."

Visão geral da *framework* de coleções Java
Interfaces – características principais

Interface	Características principais
`Set`	• representa um conjunto de elementos • não permite a repetição de elementos
`SortedSet`	• versão ordenada da coleção `Set`
`List`	• representa uma lista de elementos • permite a repetição de elementos
`Queue`	• representa uma fila de elementos • acesso aos elementos com disciplina FIFO
`Dequeue`	• *double ended queue* – equivalente a Queue mas com acesso para inserir e retirar em ambas as extremidades
`Map`	• representa um contentor associativo de pares chave/valor

Visão geral da *framework* de coleções Java
Classes abstratas

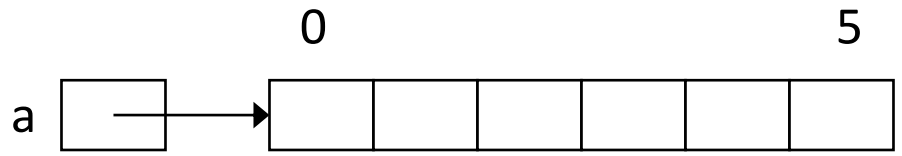
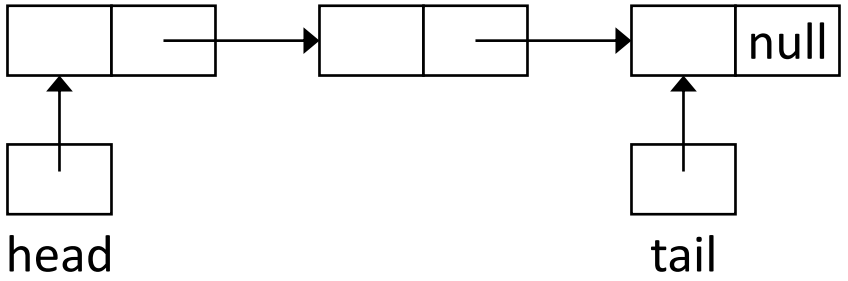
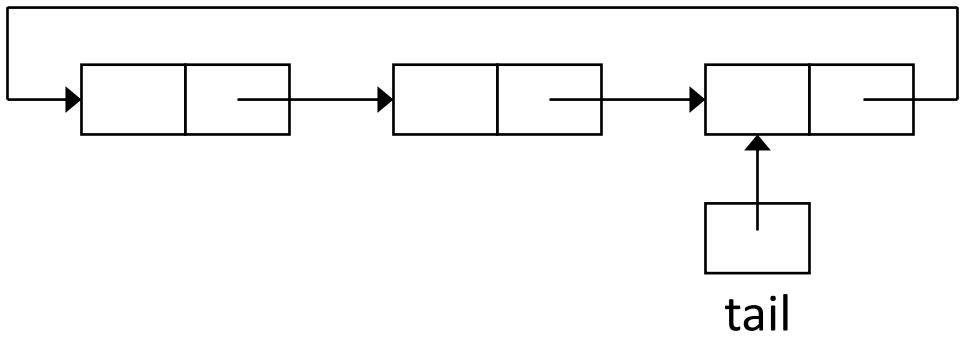
- Providenciam implementações parciais das interfaces, nomeadamente, as operações que não dependam da estrutura de dados que suporta a coleção

- Facilitam a implementação dos tipos coleção concretos mas não impedem que as operações fornecidas possam ser sobrepostas (*overriden*)

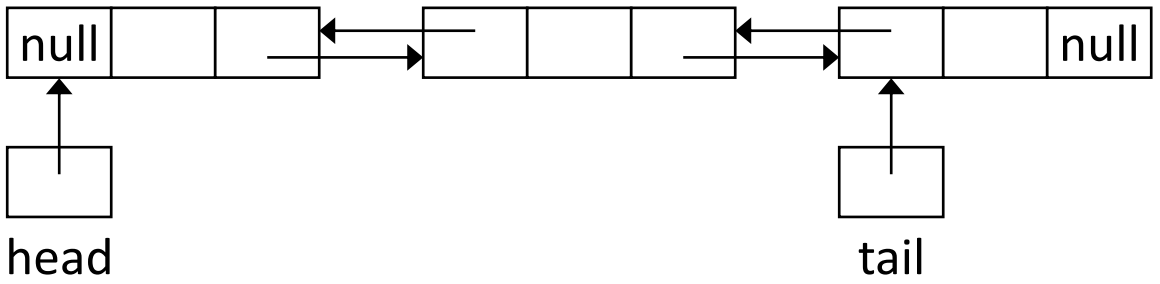
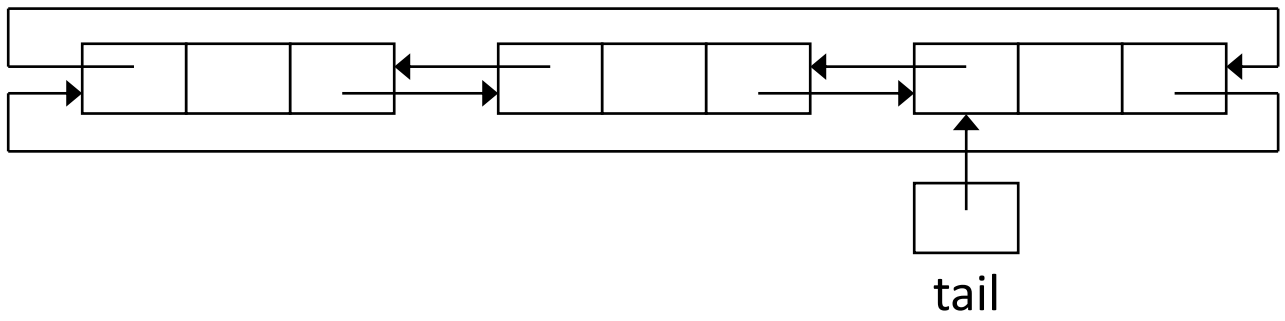
Visão geral da *framework* de coleções Java
Classes e modelo de dados subjacente

Classes concretas	Modelo de dados subjacente
`ArrayList`	• *array* com capacidade reconfigurável com acesso direto aos elementos via índice
`LinkedList`	• *lista duplamente ligada* adequada para inserções e remoções frequentes
`HashSet`	• *tabela de hash* para armazenar os elementos e não garante a ordem dos mesmos
`TreeSet`	• *árvore balanceada* para armazenar os elementos e os elementos estão ordenados de acordo com a sua ordem natural ou de acordo com o `Comparator` especificado na construção
`HashMap`	• *tabela de hash* para armazenar pares de elementos chave/valor e não garante a ordem dos mesmos
`TreeMap`	• equivalente a um `TreeSet` mas para armazenar pares de elementos chave/valor; os elementos estão ordenados pela chave, de acordo com a sua ordem natural ou de acordo com o `Comparator` especificado na construção

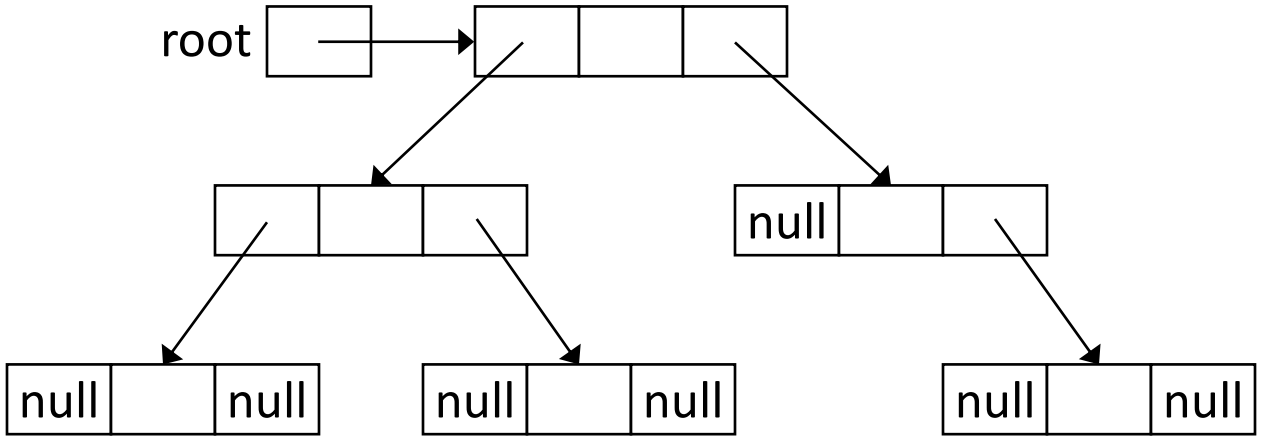
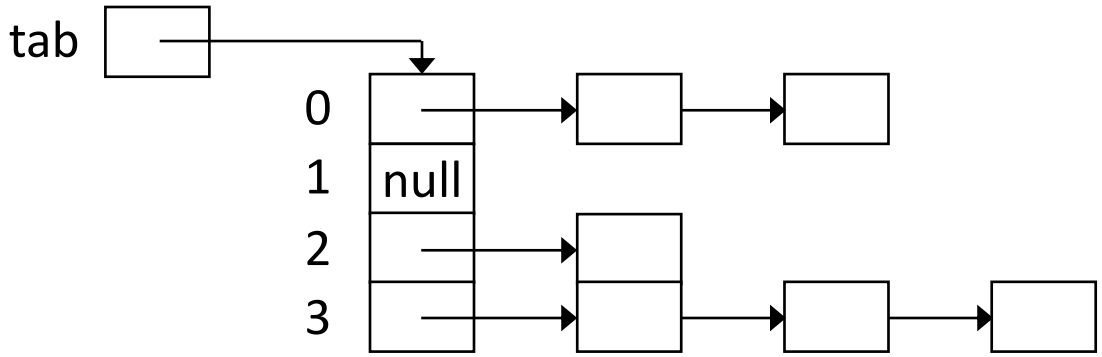
Estruturas de dados em memória (1 de 3)

Tipo de estrutura de dados	Modelo de dados em memória
Array	
Lista simplesmente ligada	
Lista simplesmente ligada circular	

Estruturas de dados em memória (2 de 3)

Tipo de estrutura de dados	Modelo de dados em memória
Lista duplamente ligada	
Lista duplamente ligada circular	

Estruturas de dados em memória (3 de 3)

Tipo de estrutura de dados	Modelo de dados em memória
Árvore binária	
Tabela de hash	

Que tipo de coleção se deve ser usar? (1 de 4)

- A decisão depende, essencialmente, de três fatores:
  1. Permite ou não repetições
  2. Aceder aos elementos por posição
  3. Aceder aos elementos por chave

Que tipo de coleção se deve ser usar? (2 de 4)

Coleção	Quando	Exemplos de cenários
`ArrayList<E>`	Para guardar uma sequência que permite repetições; acesso frequente por índice; adições sobretudo no fim	Lista de alunos de uma turma; catálogo de produtos percorrido frequentemente
`LinkedList<E>`	Para uma sequência com inserções e remoções através de um iterador já posicionado, ou operações nas duas extremidades. O acesso por índice é inadequado	Lista de tarefas em que se inserem ou removem tarefas durante um percurso com `ListIterator`; sequência de etapas editada numa posição já identificada por um iterador

Que tipo de coleção se deve ser usar? (3 de 4)

Coleção	Quando	Exemplos de cenários
`TreeSet<E>`	Para manter elementos sem repetições e ordenados, permitindo procurar vizinhos e selecionar intervalos	Conjunto de números de aluno ordenados; datas de eventos sem duplicados; conjunto de pontuações distintas em que se pretende consultar a imediatamente superior ou inferior a um valor
`PriorityQueue<E>`	Para processar repetidamente o elemento com maior prioridade, segundo a ordem natural ou um Comparator	Escalonamento de tarefas por urgência; simulação que processa o próximo evento por instante; processamento de pedidos por prioridade

Que tipo de coleção se deve ser usar? (4 de 4)

Coleção	Quando	Exemplos de cenários
`HashMap<K,V>`	Para associar chaves únicas a valores, com pesquisa, inserção e remoção eficientes em média, sem exigir uma ordem de percurso	Consultar um aluno pelo número; associar códigos de produto aos respetivos produtos; contar ocorrências de palavras num texto
`TreeMap<K,V>`	Para associar chaves únicas a valores, mantendo as chaves ordenadas, com consultas por intervalo ou proximidade	Agenda que associa instantes a compromissos; tabela de preços por escalão, consultando o escalão aplicável com `floorEntry`; registos por data com consulta de um intervalo temporal

Visão geral da *framework* de coleções Java
Classe `Collections`

- Classe equivalente à classe `Arrays` para *arrays*
- Disponibiliza um conjunto de algoritmos para coleções na forma de métodos estáticos
- Algoritmos, maioritariamente, aplicados a listas: ordenar; baralhar; pesquisar; encontrar valores extremos

Resumo

- *Framework* de coleções Java / Estruturas de dados dinâmicas

- Complexidade de um algoritmo

Complexidade de um algoritmo

- Como se mede a eficiência ou a complexidade de um algoritmo?
- Não é interessante uma medição concreta pois depende de diferentes fatores: número de elementos; ordem original dos elementos; arquitetura e sistema onde é executado, ...
- Mais interessante: “taxa de crescimento/fator de crescimento/ordem de crescimento”, portanto, qual o custo ao alterar o número de elementos da sequência
  - A complexidade exprime o tipo de proporcionalidade existente entre um conjunto de *n* elementos e o tempo *t(n)* que demora a sua execução
- Considera-se uma sequência constituída por um número elevado de elementos, favorecendo as características principais do algoritmo e desprezando custos de ordem menor (constantes)

Notação assintótica

- Big-Oh (*Ómicron*) ou Big-O: define o limite superior do fator de crescimento (*upper bound*)
  - Considera a ordem dos elementos que proporciona maior tempo de execução

  Exemplo 1:

  A pesquisa num *array* não ordenado requer, no máximo, a iteração sobre todos os elementos (***n***). Considerando ***k*** o tempo de execução de cada iteração, o tempo total de execução corresponde a ***t(n) <= k n***, ou seja, linear.

  Em notação BIG-O, escreve-se ***t(n) = O(n)***, ou seja, o tempo cresce linearmente com o tamanho da entrada

  Exemplo 2:

  A pesquisa num *array* ordenado pode usar pesquisa dicotómica, ou seja, por cada iteração elimina metade dos elementos em pesquisa. O tempo total de execução corresponde a $t(n) <= k \log_2 n$, ou seja, logarítmico.

  Em notação BIG-O, escreve-se ***t(n) = O(log n)***, ou seja, o tempo cresce logaritmicamente com o tamanho da entrada

- Este tipo de notação não pretende quantificar o tempo exato de execução do algoritmo; pretende antes caracterizar o fator de crescimento que existe com o aumento do número de elementos envolvidos no algoritmo e a respetiva execução

Outros tipos de notação assintótica

- Big-Oh (*Ómicron*) ou Big-O: define o limite superior do fator de crescimento (*upper bound*) - considera a ordem dos elementos que proporciona maior tempo de execução

- Big-Ω (*Omega*): define o limite inferior do fator de crescimento da complexidade (*lower bound*) – considera a ordem dos elementos que proporciona melhor tempo de execução

- Big-Θ (*Theta*): define um limite *apertado*; está situado entre BIG-O e Big-Ω

Por norma, a notação Big-O é a mais relevante e a mais usada porque indica o pior caso de crescimento – notação usada daqui em diante

Exemplos de complexidade com Big-O

Complexidade	Nome	Exemplo
$O(1)$	Constante	Aceder a um elemento num *array*
$O(\log n)$	Logarítmica	Pesquisa binária
$O(n)$	Linear	Iterar numa lista
$O(n \log n)$	Quase linear	`MergeSort`
$O(n^2)$	Quadrática	Dois ciclos aninhados
$O(2^n)$	Exponencial	Força bruta (experimentar todos os casos)

- Degradação da complexidade no sentido descendente da tabela
- Por exemplo, se n duplicar:
  - $O(n)$ -> demora o dobro do tempo
  - $O(n^2)$ -> demora o quádruplo do tempo
  - $O(\log n)$ -> demora log(n) mais uma constante

Comparação entre os vários tipos de complexidade

- Considerando:
  Número de elementos ***n = 10^5***
  Tempo de execução de cada passo ***k = 10^{-5}*** segundos

Complexidade $O(g(n)) \Rightarrow O(g(10^5))$	**Tempo total de execução** $k \times g(n) \Rightarrow 10^{-5} \times g(10^5)$
$O(1)$	$10^{-5} \times 1 = 1 \times 10^{-5} = 10 \times 10^{-6} = 10\ \mu s$
$O(\log_2 n)$	$10^{-5} \times log_2 10^5 \cong 10^{-5} \times 16.6 \cong 170\ \mu s$
$O(n)$	$10^{-5} \times 10^5 = 1 \times 10^0 = 1\ s$
$O(n \log_2 n)$	$10^{-5} \times 10^5 \times log_2 10^5 \cong 1 \times 16.6 \cong 17\ s$
$O(n^2)$	$10^{-5} \times 10^{5^2} = 10^{-5} \times 10^{10} = 10^5 \cong 28\ horas$
$O(2^n)$	$10^{-5} \times 2^{10^5} = 10^{-5} \times 2^{50} \cong 357\ anos$